

C++第六章

1.constexpr (编译期函数/常量)

- `constexpr` — Compile-time function
 - `constexpr` 向编译器提示：**在编译期即可求值**（无运行时开销）。
 - 典型用途：把能在编译期确定的计算（`+`，`-`，`*`，`/` 等纯函数）提前算好，减少运行期开销。
 - 过多/复杂的编译期计算会**拖慢编译**。
 - 示例代码展示了 `sum/square/factorial` 的 `constexpr` 版本；当实参是常量表达式时能在编译期折叠。

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // 编译期函数：若参数为常量表达式，可在编译期求值
6 constexpr auto sum(int x, int y, int z) { return x + y + z; }
7 constexpr int square(int v) { return v * v; }
8 constexpr int factorial(int n) { return (n <= 1) ? 1 : (n *
9 factorial(n - 1)); }
10
11 int main() {
12     // 这些都是常量表达式：编译期求值
13     constexpr auto z2 = 4;
14     constexpr float x = 42.0f + z2;
15     cout << x << "\n";
16
17     constexpr auto value = sum(3, 4, 5);
18     cout << value << "\n";
19
20     constexpr auto factorial_10 = factorial(10);
21     cout << factorial_10 << "\n";
22
23     // 演示：运行期参数->仍可调用constexpr函数，但此时在运行期求值
24     int n; cin >> n;
25     cout << "square(n) =" << square(n) << "\n";
26     return 0;
27 }
```

2.Generation of random number随机数生成

- True vs. pseudo-random number
 - 金融/仿真里随机数是**随机驱动** (stochastic) 来源。
 - **真随机**：热噪声/大气噪声/硬件 TRNG（例如 random.org）。优点：**真正随机**；缺点：**慢、不可复现**。
 - **伪随机**：由**确定性算法**生成的数列，满足统计随机性检验；优点：**可复现、可移植、快**。量化仿真通常使用伪随机而非真随机。
- Random number generator

- 推荐使用 **Mersenne Twister** (C++ 标准库里是 `std::mt19937 / mt19937_64`)。
- 速度快、周期极长：

$$\text{period} = 2^{19937} - 1$$

C++11 起, `<random>` 已取代 C 的 `rand()`。

- Seeding a random number generator

- **种子 (seed)** 决定序列起点；同样的种子→同样的序列（可复现）。
- 可用 `std::seed_seq` 用若干整数初始化状态；也可用 `std::random_device` 得到近似真随机种子（不可复现、较慢）。

可复现种子：

`seed_seq {90128, 43021, 89427} → mt19937`

```
1 // 1) 可复现的初始化: seed_seq (推荐用于固定实验)
2 seed_seq seed2{90128u, 43021u, 89427u}; // 多个32位种子更充分初始化状态
3 mt19937 gen_det(seed2); // 32位 Mersenne Twister
```

不可复现（近似真随机）种子：

`random_device rd; mt19937 gen(rd());`

```
1 // 2) 不可复现的初始化: random_device (真实硬件熵源/系统熵池)
2 random_device rd;
3 mt19937 gen_nondet(rd());
```

- 直观理解：设内部大序列为 $[s_0, s_1, \dots]$ ，给定种子选定一个起点 k ，输出序列为

$$(s_k, s_{k+1}, s_{k+2}, \dots)$$

- Define a distribution

- C++ 提供多种**分布适配器**：均匀、正态、Poisson、指数、伯努利等。
- 通过分布把生成器的“均匀比特流”映射到目标分布

$$X \sim \text{Bernoulli}(p), \quad Y \sim \mathcal{N}(\mu, \sigma^2), \quad Z \sim \text{Uniform}[a, b]$$

```
1 // 3) 定义若干分布
2 uniform_int_distribution<int> U(1, 6); // [1,6] 均匀整数 (掷骰子)
3 normal_distribution<double> N(0.0, 1.0); // 标准正态
4 bernoulli_distribution B(0.5); // 伯努利 p=0.5
5
6 cout << boolalpha;
```

```

1 // 3.1 用可复现引擎生成：每次运行一样
2 cout << "Deterministic (seed_seq) dice: ";
3 for (int i = 0; i < 6; ++i) cout << U(gen_det) << " ";
4 cout << "\nDeterministic Bernoulli: ";
5 for (int i = 0; i < 5; ++i) cout << B(gen_det) << " ";
6 cout << "\nDeterministic Normal: ";
7 for (int i = 0; i < 3; ++i) cout << N(gen_det) << " ";
8 cout << "\n\n";
9
10 // 3.2 用不可复现引擎生成：每次运行不同
11 cout << "Non-deterministic (random_device) dice: ";
12 for (int i = 0; i < 6; ++i) cout << U(gen_nondet) << " ";
13 cout << "\n";

```

- Uniform distribution

- `uniform_int_distribution(a, b)` 生成整数，覆盖区间 $[a, b]$ 。
- `uniform_real_distribution(a, b)` 生成浮点数，覆盖区间 $[a, b]$ 。

示例：

- 整数：生成 0 或 1。
- 浮点数：生成 $[0, 1)$ 内随机数，如 0.359、0.528。

```

1 // 全局随机数引擎
2 seed_seq seed2{90128};
3 mt19937 mtgen{seed2};
4
5 uniform_int_distribution<> ud(0, 1);
6 cout << ud(mtgen) << "\n"; // 第11次调用
7
8 // 均匀实数分布
9 uniform_real_distribution<> urd(0, 1);
10 cout << urd(mtgen) << "\n";

```

- Normal distribution

- `normal_distribution(mean, stddev)` 用于生成服从正态分布 $\mathcal{N}(\mu, \sigma^2)$ 的浮点数。
- 示例中：均值 $\mu = 5.0$ ，标准差 $\sigma = 2.0$ ，生成一个长度为 10 的向量。

```

1 void simulate() {
2     normal_distribution<> nd(5.0, 2.0); // 均值=5.0，方差=4.0
3     for (int i = 0; i < 10; ++i) {
4         cout << nd(mtgen) << "\n"; // 调用10次
5     }
6 }
7
8 // 正态分布，生成向量
9 vector<int> u(10);
10 auto nd = normal_distribution<>(5.0, 2.0);
11 generate(u.begin(), u.end(), [&]() { return nd(mtgen); });
12 for (auto v : u) cout << v << " ";
13 cout << "\n";

```

- Use mtgen in shuffle

- `shuffle` 可以用随机数引擎 `mtgen` 打乱容器顺序。
- 示例: `shuffle(v.begin(), v.end(), mtgen);`

```
1 vector<int> v = {1,2,3,4,5};
2 shuffle(v.begin(), v.end(), mtgen);
3 for (auto x : v) cout << x << " ";
4 cout << "\n";
```

- Make `mtgen` global variable
 - 通常不推荐使用全局变量, 但 `mtgen` 可以作为例外。
 - 这样不同函数调用共享同一个随机数序列, 保证 **复现性**。
 - 例子: `simulate()` 中调用 10 次, 主函数又调用一次 → 序列是连续的 (第 11 次)。

- Exercise 1

题目要求写一个程序:

- 生成 1,000,000 个随机整数, 范围是 [1, 2000]。
- 统计其中有多少个数落在 [500, 1000] 之间, 并且能被 163 整除。

```
1 #include <iostream>
2 #include <random>
3 using namespace std;
4
5 int main() {
6     mt19937 mtgen(random_device{}());
7     uniform_int_distribution<> dist(1, 2000);
8
9     int count = 0;
10    for (int i = 0; i < 1000000; ++i) {
11        int num = dist(mtgen);
12        if (num >= 500 && num <= 1000 && num % 163 == 0) {
13            count++;
14        }
15    }
16    cout << "Count = " << count << "\n";
17    return 0;
18 }
```

3.Application in Risk Management金融风险管理应用

Storyline - 1

- **SVB 成立背景:** 1983 年成立, 服务对象是科技、风险投资、私募股权、葡萄酒产业等新兴公司。
- **疫情期间:** 科技行业繁荣, 大量资金流入银行, 而利率极低。SVB 将资金投资于 **MBS (抵押贷款支持证券)**, 并打算持有到期 (HTM)。
- **MBS 特点:** 一种衍生品, 类似债券, 本质上是将资金借给房主。
- **利率上升:** 美联储从 2022 年 3 月的 0.25 快速加息至 2023 年 3 月的 5% 长期利率 (10 年国债收益率) 也从 1.5 升至 4。

Storyline - 2

加息引发连锁反应：

1. SVB 客户资金流入减少，部分客户为支付开销或追求更高利率而提款。
2. SVB 需要提高存款利率来留住客户，但资金已经投进了 MBS，缺乏流动性。
3. 随着利率上升，MBS 价格大幅下跌。卖出会亏损，不卖出则现金流不足。

MBS Holding in SVB

- 到 2022 年底，SVB 持有约 670 亿美元的 MBS，平均收益率约 1.5%。
- 当时美联储利率已升至 4%，MBS 收益率远低于市场利率，SVB 难以支付更高的存款利率。
- 图表展示了 SVB 的证券投资组合，其中 **大部分是低收益 MBS**。

MBS Collapse

- 2023 年 Q1 财报更新，SVB 宣布要筹集 22.5 亿美元资本。
- 原因是其出售 **“可供出售证券 (AFS)”** 时亏损 18 亿美元。
- 出售资产总额达 **\$210 亿美元**，包括美国国债和 MBS。
- 这一消息发布后，市场恐慌，引发 **银行挤兑**，导致 SVB 的快速崩溃。

Risk in Bank - Interest Rate Risk

- 银行模式：短期借款、长期放贷。如果利率曲线正常（上升），可以低成本借入、高利率贷出。
- 如果利率曲线倒挂（短端利率 > 长端利率），银行会遇到困难：高利率借入、低利率贷出。
- 从 **2020 疫情后利率极低** → **2023 快速加息**，利率环境由正斜率转为倒挂，SVB 未能调整资产或做风险对冲，暴露出 **利率风险**。
- 银行风险经理的日常问题：如果资产价格下跌而银行无法持有至到期，会造成亏损。
- 长期固定收益投资（如 MBS）对利率非常敏感，持仓过大 → 利率上升时风险显现。
- 衡量利率风险的方法：**Duration（久期）** 和 **Convexity（凸性）**。

Duration (久期)

久期是衡量债券对利率变动敏感度的核心指标：

1. **Macaulay Duration**（麦考利久期）：加权平均现金流时间（以年为单位）。
2. **Modified Duration**（修正久期）：价格相对变化率与利率变化的关系（百分比）。
3. **Effective Duration**（有效久期）：数值近似法，考虑到利率变化对价格的影响。

公式：

Macaulay Duration：加权平均现金流时间

$$\text{Macaulay Duration} = \sum_{i=1}^n t_i \frac{PV_i}{V}$$

Modified Duration：修正久期衡量价格对利率变化的敏感度：

$$\text{Modified Duration} = \frac{\text{Macaulay Duration}}{1 + \frac{YTM}{k}}$$

Effective Duration：数值近似法：

$$\text{Effective Duration} \approx \frac{P^- - P^+}{2P_0\Delta Y}$$

其中 P^- 是利率下降后的债券价格, P^+ 是利率上升后的债券价格, P_0 是初始价格。

On-the-back envelope calculation

- MBS (抵押贷款支持证券) 的平均久期大约是 6 (截至 2023 年 2 月)。
- 如果 10 年期国债利率上升 2.5%, 则 MBS 价格大约下跌 6×2.5 。
- 这和 SVB 卖出 MBS 组合时大约 -10 的损失 (1.8B/21B) 相符。
- 但要注意, SVB 卖出的是久期更短的资产, 剩下的长期资产潜在风险更大。
- Bond Convexity
 - 久期 (Duration) 衡量的是一阶敏感度 (线性近似)。
 - 凸性 (Convexity) 衡量的是二阶效应, 即债券价格曲线与利率的二次关系。
 - 凸性公式:

债券的凸性定义为:

$$\text{Convexity} = \frac{1}{P(1 + \frac{YTM}{k})^2 k^2} \sum_{i=1}^n PV_i(i^2 + i)$$

数值近似的有效凸性为:

$$\text{Effective Convexity} \approx \frac{P^- + P^+ - 2P_0}{P_0(\Delta Y)^2}$$

- Taylor 展开公式

结合久期与凸性, 债券价格的变化公式为:

$$\frac{\Delta P}{P} \approx -\text{Duration}_{\text{Mod}} \cdot \Delta Y + 0.5 \cdot \text{Convexity} \cdot (\Delta Y)^2$$

其中 P 为债券现值, ΔY 是收益率变化, ΔP 是价格变化。

- 债券数据结构
 - 使用 `struct` 定义一个固定利率债券 (FixedRateBond), 包含:
 - 面值 `face_value`
 - 息票率 `coupon_rate`
 - 到期时间 `maturity`
 - 支付频率字符串 `ps`

```
1 struct FixedRateBond {
2     double face_value;
3     double coupon_rate;
4     double maturity;
5     string ps;
6     double ytm;
7 };
```

- 实现函数 `bond_price_ytm`

- 输入：一个债券对象
- 计算：

- 现值 (PV)
- Macaulay 久期
- 修正久期 (Modified Duration)
- 凸性 (Convexity)

使用公式：

$$PV = \sum_{i=1}^n \frac{CF_i}{(1 + ytm/m)^i}$$

$$MacaulayDuration = \frac{\sum t_i \cdot \frac{CF_i}{(1 + ytm/m)^i}}{PV}$$

$$ModifiedDuration = \frac{MacaulayDuration}{1 + ytm/m}$$

$$Convexity = \frac{\sum CF_i(i^2 + i)}{PV(1 + ytm/m)^2 m^2}$$

```
1 auto bond_price_ytm(FixedRateBond frb) {
2     int payment_count = 1; // 假设每年付息一次
3     int n = frb.maturity * payment_count;
4     vector<double> cf(n, frb.coupon_rate * frb.face_value /
5         payment_count);
6     cf.back() += frb.face_value;
7     vector<double> df(n, 0.0);
8     double pv = 0.0, dcf = 0.0, mac_duration = 0.0, convexity = 0.0;
9     double unit_rate = 1.0 + frb.ytm / payment_count;
10
11     for (int i = 0; i < n; ++i) {
12         df[i] = 1.0 / pow(unit_rate, i + 1);
13         dcf = df[i] * cf[i];
14         mac_duration += dcf * (i + 1) / payment_count;
15         convexity += dcf * (i + 2) * (i + 1);
16         pv += dcf;
17     }
18
19     mac_duration /= pv;
20     convexity /= pv * pow(unit_rate, 2) * pow(payment_count, 2);
21     double mod_duration = mac_duration / unit_rate;
22
23     return make_tuple(pv, mac_duration, mod_duration, convexity);
24 }
```

• 测试部分

- 定义了 isclose() 判断两个浮点数是否近似相等。
- 输出测试结果：债券现值、久期、凸性，验证公式的正确性。
- 测试通过，说明实现正确。

```
1 // 浮点数近似比较
2 bool isclose(double a, double b, int power = -7) {
```

```

3     return fabs(a / b - 1) < pow(10, power);
4 }
5 /*
6  输出结果:
7  // Test for bond price and ytm
8  test_print_bond_price_ytm({100, 0.06, 30, "A", 0.06}); // pv ≈ 100
9  // Test convexity
10 // Test mac_duration
11 // Test for modified duration and convexity:
12 bond: 100, coupon: 0.02, maturity: 5, ps: A, ytm: 0.02
13 pv: 100, mac: 4.807728699, mod: 4.713459509,
14 convexity: 27.36025976
15 pv_diff: 89.02505814-100=-10.97494186
16 approx_mod: -0.1178364877, approx_mod_convexity:-0.1092864065,
17 approx_pv: -10.9286406538, diff: 0.0463012072%
18 Test for modified duration and convexity: PASS
19 */

```

- 风险解码器 (Pricing as a risk decoder)
 - 银行的风险系统会将各种金融产品（贷款、固定利率债券、浮动利率债券、MBS 等）分解为“风险因子”，例如：
 - Duration
 - Convexity
 - IR Delta/Gamma/Vega（期权风险指标）
 - 这些风险因子就是价格对市场因子的 1 阶、2 阶导数（主要是利率曲线）。
 - 管理者可以据此感知风险大小，并通过对冲工具（债券远期、利率互换 IRS）来对冲风险。
 - 衍生品能给懂的人带来收益和风险管理的能力，而对不懂的人则可能带来损失。
- MBS 模拟为固定利率债券
 - MBS 背后是住房抵押贷款，房主向银行还款，银行再把现金流打包卖给投资者。
 - 在美国，常见的是 **30 年期固定利率贷款**，但房主可以每隔 5 年重新贷款（refinance），这会提前还款（prepayment）。
 - 提前还款风险意味着 MBS 投资人可能收不到预期的利息。
 - 因此可以用 **5 年期固定利率债券** 来模拟 MBS 的现金流。

```

1 #include <iostream>
2 #include <tuple>
3 using namespace std;
4
5 // 假设已经实现了 bond pricing 函数 test_print_bond_price_ytm
6 // 这里只展示 MBS 模拟调用
7 int main() {
8     cout << "test for MBS simulated" << "\n";
9     FixedRateBond frb{100, 0.02, 5, "M", 0.02}; // 面值100, 息票率2%, 期限5年
10    auto pv = get<0>(test_print_bond_price_ytm(frb));
11    cout << "pv:" << pv << "\n";
12 }

```

- MBS 行为图示

- 如果利率 **下降** → 房主提前还款，MBS 的实际期限会变短。
- 如果利率 **上升** → 房主不提前还款，MBS 的实际期限会拉长。
- 因此 MBS 的现金流表现 **不像固定久期债券**，而是随利率环境变化。

• Simulate MBS with fixed-rate bond - 2 (利率下降)

利率↓ ⇒ 住户更可能再融资 ⇒ **提前还款上升** ⇒ 以“**缩短到期 + 降低 YTM**”来近似 ⇒ MBS 价格略涨，但涨幅小于同久期不变的 FRB。

Simulate MBS with fixed-rate bond - 3 (利率上升)

利率↑ ⇒ 不会再融资 ⇒ **延长期限** ⇒ 以“**拉长到期 + 提高 YTM**”来近似 ⇒ MBS 价格**远跌于**久期不变的 FRB。

Negative convexity for MBS (负凸性)

利率↓的涨幅小、利率↑的跌幅大；与线性久期给出的对称近似不同，这就是**负凸性**（对冲更难）。

Pricing as a tool for stress test

定义一组情景 (what-if)：速变/渐变/突发；用定价引擎逐一计算 PV/敏感度，检验脆弱点。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // --- 基础：固定利率债券定价 + 久期/凸性 ---
5  struct Bond { double face, coupon, maturity; double ytm; int m=1; };
6
7  struct PVDC {
8      double P, D_mod, C; // 价格、修正久期、凸性
9  };
10
11  PVDC price_duration_convexity(const Bond& b){
12      int n = int(round(b.maturity * b.m));
13      vector<double> cf(n, b.coupon * b.face / b.m);
14      if(n==0){ return {b.face, 0, 0}; }
15      cf.back() += b.face;
16
17      const double r = b.ytm / b.m;
18      const double u = 1.0 + r;
19
20      double P=0, Mac=0, Cnum=0;
21      for(int i=1; i<=n; ++i){
22          double df = pow(u, -i);
23          double c = cf[i-1];
24          P += c * df;
25          Mac += i * c * df;
26          Cnum += (i*(i+1)) * c * df;
27      }
28      Mac /= P;
29      double D_mod = Mac / u;
30      double C = Cnum / (P * u * u * b.m * b.m);
31      return {P, D_mod, C};
32  }
33
34  // --- 工具：打印对比 ---
35  void print_line(const string& tag, const PVDC& x){
36      cout << setw(14) << left << tag
37          << " P=" << x.P

```

```

38     << "    Dmod=" << x.D_mod
39     << "    C=" << x.C << "\n";
40 }
41
42 // --- 情景: 用债券近似 MBS ---
43 PVDC mbs_like_price(double face, double coupon, int base_year, double
base_ytm,
44                     double dY, int dMat){
45     Bond b{face, coupon, double(base_year + dMat), base_ytm + dY, 1};
46     return price_duration_convexity(b);
47 }
48
49 // --- 主程序: 下降/上升情景 + 负凸性检查 + 压力测试骨架 ---
50 int main() {
51     ios::sync_with_stdio(false);
52     cout.setf(std::ios::fixed); cout<<setprecision(6);
53
54     // 基准: 5y, 2% 息票, 2% YTM (把它当作“FRB”与“MBS 的基线”)
55     Bond base{100.0, 0.02, 5.0, 0.02, 1};
56     auto base_pdc = price_duration_convexity(base);
57     print_line("Base(FRB)", base_pdc);
58
59     // --- 情景 A: 利率下降 (近似提前还款 -> 缩短 1y; 同时 YTM 下调 1%) ---
60     auto mbs_down = mbs_like_price(100, 0.02, 5, 0.02, -0.01, -1); //
4y, 1%
61     auto frb_down = price_duration_convexity(Bond{100,0.02,5,0.01,1});
62     // 5y, 1%
63     cout << "\n[Rate Down]\n";
64     print_line("MBS_down", mbs_down);
65     print_line("FRB_down", frb_down);
66     cout << "diff_P(MBS-base) = " << (mbs_down.P - base_pdc.P) << "\n";
67     cout << "diff_P(FRB-base) = " << (frb_down.P - base_pdc.P) << "\n";
68
69     // --- 情景 B: 利率上升 (不再再融资 -> 延长 5y -> 约 10y; YTM 上调 1%) ---
70     auto mbs_up = mbs_like_price(100, 0.02, 5, 0.02, +0.01, +5); //
10y, 3%
71     auto frb_up = price_duration_convexity(Bond{100,0.02,5,0.03,1}); //
5y, 3%
72     cout << "\n[Rate Up]\n";
73     print_line("MBS_up", mbs_up);
74     print_line("FRB_up", frb_up);
75     cout << "diff_P(MBS-base) = " << (mbs_up.P - base_pdc.P) << "\n";
76     cout << "diff_P(FRB-base) = " << (frb_up.P - base_pdc.P) << "\n";
77
78     // --- 负凸性“经验性”检查: |跌幅(MBS)| > |涨幅(MBS)|
79     double up_loss = base_pdc.P - mbs_up.P; // 利率上升 -> 跌
80     double dn_gain = mbs_down.P - base_pdc.P; // 利率下降 -> 涨
81     cout << "\n[Neg Convexity check]\n";
82     cout << "gain_down=" << dn_gain << "    loss_up=" << up_loss << "\n";
83     if (up_loss > dn_gain) cout << "MBS shows NEGATIVE CONVEXITY.\n";
84     else cout << "No negative convexity in this toy
setup.\n";
85
86     // --- 压力测试骨架: 可扩展多情景 ---
87     struct Scenario{ string name; double dY; int dMat; };
88     vector<Scenario> scen = {

```

```

88     {"mild_down", -0.005, -1},
89     {"shock_up", +0.015, +5},
90     {"gradual_up", +0.010, +3},
91 };
92 cout << "\n[Stress Test: PV changes]\n";
93 for (auto s: scen){
94     auto x = mbs_like_price(100,0.02,5,0.02,s.dy,s.dMat);
95     cout << setw(12) << left << s.name
96         << "   dP=" << (x.P - base_pdc.P)
97         << "   Dmod=" << x.D_mod
98         << "   C=" << x.C << "\n";
99 }
100 return 0;
101 }

```

- What SVB Management Could Do
 - **减少对 MBS 的过度集中**，增加投资多样性。
 - **增加对冲**，但难点在于规模和负凸性。
 - **改变会计规则**：HTM（持有至到期）未市值计量，导致风险滞后暴露。
 - **向美联储求助**：自身难以扭转局面。
 - **CRO 离职**：SVB 自 2022 年 4 月起缺少首席风险官。

The Risk Management System

- 金融机构依赖**风险管理系统**来记账、接收数据流、计算风险。
- 系统要求：**数值正确、可靠、易用**。

The Risk Management System – Key Output

- 实时计算一阶/二阶风险敞口（风险大小与速度）。
- 压力测试（stress testing）揭示未来风险。
- VaR 或预期短缺（Expected shortfall），作为风险资本。
- 交易对手风险（Counterparty risk 与 xVA）。

Software for Complex System

- 实际上有**大量金融工具、数据、风险指标、市场情景**需要统一处理。
- 需要强大软件来应对复杂性。
- 在 OOP I 学会解决具体问题；在 OOP II 将学习如何应对复杂系统。

- Practical tips
 - 开发流程：**设计 → 写代码 → 测试**
 - 如果不知道如何开始，可以先写测试，帮助明确目标。
 - 尽快写出一个能运行的版本，哪怕功能不完整。
 - 先写结果固定的函数，逐步扩展。
- Guidelines
 1. 禁止全局变量（除随机数生成器 `mt19937` 或 `NNdistribution` 外）。
 2. 必须使用 C++ 标准库（禁止 C 风格头文件）。
 3. 使用 STL 容器，禁止 C 风格数组。
 4. 类型转换使用 `staticcast`。

5. 使用 *double*, 不要用 *float*。

6. 合理使用 *auto* 简化代码。

7. 不要盲目强制赋零。

8. 鼓励用 STL 算法替代 *for* 循环。